

QOTP – Why: the reasoning behind the mechanisms

The specification says what a second implementation must agree on. This document says why the reference implementation does what it does, so that a reader can judge which choices are essential and which are ours. Section numbers here are A.n; the specification's are §n. Numbers are the reference implementation's defaults.

A.1 One suite, two layers

There is no cipher negotiation because negotiation is where downgrade attacks live, and X25519 with ChaCha20-Poly1305 has no known weakness that would force a change within the life of version 0. If one appears, the version bits select a successor.

The sequence number is encrypted separately, under a nonce taken from the sealed payload, so that a passive observer cannot count packets or correlate flows by their counters. QUIC solves the same problem with header protection; this is the same idea with fewer moving parts.

A.2 Padding and the connection ID

Both initiating messages are padded to 1232 bytes so that a small forged packet cannot make a server send a larger reply to a victim. 1232 is the payload that fits the IPv6 minimum link MTU of 1280, so the padded packet itself is never fragmented.

The connection ID is the first 8 bytes of the initiator's ephemeral key: unique enough for one listener, free to compute, and it lets a receiver find the key before it decrypts.

A.3 Path MTU

A connection starts at 1232 and rises to the smaller of the two endpoints' advertised sizes. That size is what the interfaces support, not what the path carries: a tunnel or a PPPoE link in between may be smaller, and with Don't Fragment set such a link drops the packet and often nobody hears the ICMP error. Full-size packets then vanish while small ones pass.

The reference implementation detects that without probe traffic. A packet that has failed repeatedly is retransmitted at 1232 for its last two attempts. If one of those is acknowledged while no full-size first transmission ever was, the working size was never viable on this path, and the connection drops to 1232 for its whole life. A size that was once confirmed by an ACK is never downgraded: after that, loss is congestion, not an MTU problem. Only first transmissions confirm a size, because an ACK for a retransmit could answer any of the sends.

A.4 What rides on every packet

maxPayload and rcvWnd are unconditional so that a changed MTU or a drained buffer reaches the peer on the next packet, with no "already announced" state that could be lost with the packet carrying it. The window in particular describes the connection, so it cannot live inside a per-stream ACK block: a sender blocked on a stale window has nothing to acknowledge, and would never receive the update.

A window is a snapshot of free space, valid at the moment it was built. Packets reorder, so an older packet can arrive after a newer one with a smaller, stale value; applying it would block the sender for nothing. The sequence number orders them.

Free space is not monotonic, so a stale value must be rejected rather than merely ignored. An absolute byte limit, as QUIC carries in MAX_DATA, is monotonic and so safe to retransmit: a late copy can never shrink the window. The reference implementation keeps free space and the sequence number that dates it, trading a retransmittable update for a one-byte field instead of an offset, and the sender's probe (A.5) instead of a retransmitted announcement.

Reliability rides in bit 31 of the stream ID for the same reason: every packet of the stream carries it, and a best-effort stream never retransmits anything, so a flag sent once could be lost forever.

A.5 A full receiver

With an honest window the sender never overruns: it stops when one more packet would not fit. Its own accounting is conservative, since a byte the receiver has stored but not yet acknowledged counts on both sides at once. So in the normal case nothing is ever dropped; "full" means the sender waits at the edge.

Waiting at the edge is a deadlock unless someone speaks. The window only travels on packets, and a peer that receives nothing has nothing to acknowledge. The sender is the one that knows it is blocked, so the sender probes: an empty packet with a stream header, which is acknowledged like any other, and every ACK carries the window. The receiver cannot take that job over, because from its side a silent peer might be blocked or might simply have nothing to say, and announcing forever to every idle peer is waste.

The probe repeats once per RT0 with no backoff. A backoff would only reduce traffic to a peer that stays full, at the cost of slower recovery from a lost announcement: after a long block the interval would sit at two seconds. Once per RT0 caps that cost at one RT0, and the traffic it costs is a few tiny packets per second, each answered by an equally tiny ACK. The probe never gives up, because a peer refusing data is behaving correctly; a peer that has gone away is ended by the 30-second read deadline, and the probe cannot keep it alive since it sends no data.

The receiver announces a reopened window itself, so the common case costs one round trip rather than waiting for the next probe. The announcement is a single untracked packet: tracking it would occupy the stream's one zero-payload slot and give the receiver a way to fail the connection over a window update. The probe is the retry. The announcement waits until the buffer has drained by two MTUs past what was last advertised, the classic silly-window rule, or a slowly reading application would generate a packet per read.

A packet that does not fit can still arrive, from a peer that ignores the window or in the MTU corner case of A.7. The receiver accepts it anyway if it is in order, because in-order data is drainable at once and refusing it would leave a hole nothing could fill. Out-of-order data is dropped, counted, and answered with an immediate window announcement, since the peer evidently believes the window is larger than it is.

The dropped packet is never acknowledged, so the sender retransmits it. Counted naively, five such retransmits into a full buffer would end the connection after about five seconds of the receiving application not reading. So while the window is closed the sender retransmits only its lowest unacknowledged offset. Everything below that offset is acknowledged, so that packet is the receiver's next in-order byte, or a duplicate whose ACK was lost, and either way it is accepted. Every packet above it is held with its timer and attempt count intact until the window opens. The lowest offset is at the head of each generation's map (A.6), so this costs one comparison per generation and no search.

A.6 Retransmission

Packets in flight live in one map per retransmission generation, indexed by how often the packet has been sent. Within a generation every packet has the same backoff, so each map is in expiry order as well as send order, and only the heads need checking. One map in send order would put a just-retransmitted packet, with its longer timer, in front of packets that expire sooner, and fast retransmit could not reach a packet that is not at the head. The generations

cost about forty lines of trivial loops and remove every special case.

A packet is declared lost once three later packets have been acknowledged, QUIC's threshold, inherited from TCP's three duplicate ACKs. Acknowledgements of retransmitted packets give no such evidence and no RTT sample: they could answer any of the sends (Karn).

Five attempts, doubling from the RTT and capped at two seconds. The final wait is one RTT, not the backed-off interval: there is no further retransmit to space out, and backing off there would only add sixteen RTTs to the time a dead path takes to surface.

Best-effort packets, unreliable data and pings, are never retransmitted. They are dropped from tracking one RTT after sending, when the ACK is not coming, so they stop counting against the window.

A stream has one zero-payload slot per send offset, because the ACK identifies a packet by offset and length, and every empty packet at one offset would share a key. A ping, a window probe and a FIN contend for it; the FIN wins, and a probe whose ACK was lost must not hold it, or the recovery path would block itself.

A.7 Overlapping segments

The sender keeps plaintext in flight and encrypts on every send. Resending the identical ciphertext would be safe: the same nonce with the same plaintext reveals nothing new, and every field that went stale, the piggybacked ACK, the window, a key update, is already handled as a duplicate or a stale snapshot. What prevents it is the MTU: a downgrade splits a packet in flight into two smaller ones, and ciphertext cannot be split.

Those splits are the only source of overlapping segments at the receiver, and only if the original arrives after the split parts, which on a path that swallowed the original is not a real case. The receiver therefore does not reconcile overlaps on arrival. It stores segments as they come and skips, on delivery, whatever an earlier segment already covered. Between honest, authenticated peers overlapping bytes are identical.

A.8 Round-trip time

Smoothed RTT and its variation are RFC 6298. The minimum is a time-windowed minimum over ten seconds, kept as a staircase of candidates ascending in both age and value, so that when the minimum expires the smallest sample still in the window takes over. The alternative, one value reset to the current sample at expiry, takes an inflated sample whenever a standing queue exists at that moment, and the floor then ratchets upward every ten seconds. BBR avoids that with a dedicated drain phase; the staircase avoids it without one.

A packet is stamped with the time before the write, not after. A stamp taken after the syscall can be late by a scheduling pause, which makes the RTT sample short, and a minimum filter keeps a short sample for its whole window. Erring early only inflates a sample, which the filter discards.

A.9 Bandwidth

Each packet carries a snapshot of the acknowledgement state at the moment it was sent: bytes acknowledged so far, when the latest ACK arrived, and when the packet that ACK acknowledged had been sent. Its own ACK then yields a sample over the interval since: all bytes acknowledged in that interval, which averages over a flight rather than over one inter-ACK gap, divided by the longer of the ACK-side and send-side intervals. A rush of backlogged ACKs compresses the ACK interval, but those ACKs belong to older packets and stretch the send interval by the same amount, so the larger of the two keeps every sample at or below the true rate. That matters because the filter is a maximum: it forgives low samples and latches onto high ones.

The maximum is over the best sample of each of the last ten packet-timed rounds. It rises immediately and decays only as rounds retire. Rounds paced below the link rate, throttled or draining, are not retired: they measure the reduced rate, and retiring them would decay the estimate to the policy level and lower pacing further, a self-clamp seen live on short paths.

A packet with no payload gives an RTT sample but never a bandwidth sample. It went out because there was nothing else to send, so its rate measures the sender's idleness. Everything counts wire bytes, the unit pacing spends, so the estimate and the pacer cannot drift apart.

A.10 Pacing

The sender never bursts; it spaces packets to a rate. The rate is the bandwidth estimate times a gain that depends on state, in the manner of BBR: startup grows at 2.885x per round, the classic 2 over $\ln 2$, until three rounds in a row fail to grow by a probe's worth; steady state paces at 1.0x; every eight minimum-RTTs one round probes at 1.25x and the next drains at 0.75x, so a probe that finds nothing leaves no queue behind.

A standing queue is detected from delay, independently of the estimator that may have caused it: when the smoothed RTT exceeds the minimum by more than the larger of a quarter of the minimum and 5 ms, the sender drains until it falls back. The quarter is one probe round's overshoot, so a probe cannot trip its own detector. The 5 ms floor is the delay an AQM such as fq_codel or CAKE holds by design; on an 11 ms path that target alone is 44% of the minimum, and without the floor a healthy flow drained 27% of the time.

Before the first bandwidth sample the sender paces ten packets per RTT, TCP's initial window, assuming the default RTT as the RTT until the first sample. A token bucket ten packets deep lets a late wakeup send a short burst instead of losing the slots: the loop sends one packet per wakeup and wakeups quantize to about a millisecond, so scheduling from "now" would cap throughput at one packet per wakeup.

A.11 Fairness

Delay-based pacing does not yield to loss-based flows: a CUBIC neighbour backs off on every loss while the pacer keeps sending at its measured rate, and the neighbour keeps yielding. The only signal that reveals the neighbour is the loss it shares with us, so a separate multiplier listens to loss.

Loss is judged per window of eight rounds, never per round: detections cluster into the round where their gap evidence completes, and retransmissions shrink the denominator. A window does not close until it holds enough packets for the 2% threshold to amount to one loss per round, 400 packets, so a slow path extends the window in time instead of never acting. A congested window multiplies the throttle by 0.75, a clean one by 1.25, capped at 1.0 and floored at 0.3 so the flow stays alive and keeps measuring. The same normal, probe, drain principle as pacing, per window instead of per round, with its own gains. Loss during startup ends startup instead of throttling: startup finds the ceiling by growing until the path objects, so that loss is the sender's own probe.

Packets that were already in flight when the sender last responded do not count as new evidence, RFC 6582's rule, so one burst causes one reduction.

A.12 Best-effort streams

Lost data on a best-effort stream is skipped after the head-of-line gap has been open for the gap timeout, 100 ms by default and tunable per stream. This is not a jitter buffer: in-order data is never delayed, and the timer runs only while a gap is open. It bounds the stall after a loss. Data arriving below the delivery point on such a stream is counted as late without any history of what was skipped. The sender never retransmits best-effort data, so such an arrival cannot be a duplicate of what was delivered: it is

a reordered packet for a range already given up on. Drops for lack of buffer space are counted separately, since on a best-effort stream they are lost for good.

A.13 Key rotation

At sequence number 2^{46} the sender starts a rotation and at 2^{47} completes it, half a counter's width apart, so that even a peer that sees the announcement late has room to answer before the nonce would wrap. The pending announcement rides on a packet once per RT0 until acknowledged; the acknowledgement needs no timer, because a lost one is re-triggered by the re-sent announcement.

A.14 One loop

All protocol state belongs to one event loop and is touched without locks. Application goroutines reach the connection only through the send and receive buffers, each behind its own mutex, and through a few atomics: the stream close flags, the counters, and the per-stream settings the application may change while the loop runs. That is the whole concurrency model, and it is why the loop can be reasoned about as sequential code.

That one loop is also the throughput ceiling. Every connection on a listener is served by it, so a single core is the limit however many connections share it, and each data packet costs a second packet operation for its acknowledgement. In userspace this is the dominant cost at high rates, where TCP does the same work in the kernel with segmentation offload: a benchmark that pins a core is measuring per-packet cost, not the protocol. The trade is deliberate. A lockless core that reads as sequential code is worth more at this stage than line rate, and throughput grows by adding loops - more listeners, or connections sharded across them - rather than by threading one.

A.15 Where the timer and buffer numbers come from

The values not tied to a mechanism above. A second implementation may change any of them; these are the reference implementation's, with their provenance.

	enough to forget a rate that is gone
send and receive buffers, 16 MB	cap the advertised window and the data in flight; enough to fill a 1 Gbit path to about 130 ms RTT
reorder gap, 100 ms	a few typical RTTs: catches ordinary reordering, bounds the stall after a lost best-effort packet (A.12)

value	why this one
initial RT0, 200 ms	the estimate before any RTT sample; RFC 6298's 1 s would stall the first loss recovery on a fast path
RT0 floor, 100 ms	below it, timers fire on jitter rather than loss
RT0 ceiling, 2 s	bounds the backoff so a dead path is declared in seconds; unbounded doubling would reach a minute
retransmits, 5	with doubling and the ceiling, give up in 3 to 12 s by RTT: long enough to ride out a blip, short enough to fail over
probe and loss window, 8 RTTs	the length of BBR's ProbeBW gain cycle, reused as the window over which loss is judged (A.11)
min-RTT trusted, 10 s	BBR's RTProp window; a minimum older than this may belong to a path that has since changed
bandwidth filter, 10 rounds	BBR's BtlBw window: long enough to span a probe-drain cycle, short